

GearGuard: Arsitektur Aplikasi Seluler Agnostik-Detektor untuk Pemantauan Kepatuhan Alat Pelindung Diri (APD)

Ahmad Fauzan Prayogi (penulis tunggal)
github.com/afprayogi

Abstrak—Memastikan pekerja mengenakan alat pelindung diri (APD) yang diwajibkan sebelum memasuki area kerja adalah tugas keselamatan rutin yang rawan kesalahan. Penelitian yang ada banyak berfokus pada model deteksi, sedangkan alur kerja di sekitarnya—mengidentifikasi pekerja, menerapkan aturan lokasi, mencatat hasil, dan menyusun bukti audit—kurang mendapat perhatian. Makalah ini memaparkan GearGuard, aplikasi seluler lintas-platform yang mencakup alur tersebut secara menyeluruh. Kontribusi kami: (i) definisi formal kepatuhan per-scan dan per-pekerja yang tidak bergantung pada detektor tertentu, (ii) arsitektur berlapis dengan satu antarmuka yang memisahkan model pembelajaran mesin dari antarmuka pengguna, penyimpanan, dan pelaporan, serta (iii) implementasi acuan sumber terbuka berbasis Flutter dengan fitur scan, absensi, peringkat, arsip, dan laporan PDF, yang diverifikasi dengan 38 tes otomatis (cakupan baris 75%, 20 pemeriksaan aksesibilitas). Uji awal model YOLO penulis mendeteksi kelas perlengkapan yang diharapkan pada 11 dari 11 foto validasi (sekitar 80 ms per frame pada CPU desktop), sementara eksperimen ketahanan dan citra negatif menyingkap kelemahan terhadap oklusi (5/11 dengan tambalan 25%) dan warna berdaya-lihat-tinggi (*Vest* palsu pada oranye polos). Studi kebergunaan masih menjadi pekerjaan lanjutan.

Kata kunci—alat pelindung diri, keselamatan kerja, aplikasi seluler, deteksi objek, arsitektur perangkat lunak, Flutter

I. PENDAHULUAN

Lokasi konstruksi, manufaktur, dan logistik mewajibkan pekerja memakai helm, rompi reflektif, sarung tangan, dan sepatu keselamatan. Pemeriksaan biasanya dilakukan supervisor di gerbang sehingga lambat, tidak konsisten, dan tanpa catatan terstruktur. Deteksi APD berbasis visi telah banyak diteliti [1], [2], [5] dengan memanfaatkan detektor satu-tahap waktu-nyata seperti keluarga YOLO [3], [4]. Namun detektor saja belum menjadi alat keselamatan yang dapat dipakai: supervisor juga perlu tahu *siapa* yang dipindai, aturan *mana* yang berlaku, dan dapat menelusuri riwayat untuk audit.

Kontribusi: (1) formalisasi kepatuhan yang bebas-detektor; (2) arsitektur berlapis yang memperlakukan detektor sebagai komponen yang dapat diganti; (3) implementasi sumber terbuka beserta verifikasinya.

Penelitian terdahulu melaporkan akurasi deteksi pada citra lokasi kerja [1], [2], [5]; alur kerja di sekitarnya (identitas pekerja, aturan lokasi, pencatatan, laporan audit) jarang menjadi objek kajian. Makalah ini mengisi celah tersebut

dengan implementasi acuan yang menjaga detektor tetap dapat diganti.

II. TINJAUAN PUSTAKA

Deteksi APD. Wang dkk. [1] merilis dataset CHV (1.330 citra lokasi konstruksi) dan membandingkan delapan detektor berbasis YOLO untuk helm empat warna, orang, dan rompi. Karlsson dkk. [2] membangun sistem YOLOv4 untuk lima kategori perlengkapan (helm, rompi, sarung tangan, kacamata, pelindung telinga) pada titik kontrol masuk, skenario yang paling dekat dengan penelitian ini. Ordick dkk. [5] melatih YOLOv11s pada 589 citra (1.425 instans, tujuh kategori) untuk pekerja kelistrikan dan melaporkan presisi 94,0%, recall 90,1%, dan mAP@50 92,8%, dengan aplikasi web yang memproses satu citra dalam 2–3 detik pada CPU; mereka mencatat set validasi yang kecil dan sulitnya mendeteksi sepatu keselamatan.

Keluarga detektor. Detektor satu-tahap seperti YOLOv7 [3] dan YOLOv8 [4] menyeimbangkan akurasi dan latensi dan lazim dipilih untuk inferensi pada perangkat.

Celah. Karya-karya tersebut mengoptimalkan atau mengevaluasi model. Lapisan aplikasi yang mengubah deteksi menjadi keputusan dan catatan (aturan kepatuhan, agregasi per pekerja, absensi, dan keluaran audit) tidak dirinci. GearGuard memformalkan lapisan itu (Bagian III) dan mengisolasi model di balik satu antarmuka (Bagian IV-B).

III. PERUMUSAN MASALAH

Misalkan P himpunan jenis APD yang didukung (helm, rompi, sarung tangan, sepatu, kacamata, masker). Sebuah lokasi menetapkan himpunan wajib $R \subseteq P$, $R \neq \emptyset$, dan ambang keyakinan $\tau \in [0,1]$. Untuk satu frame, detektor mengembalikan keyakinan $c_p \in [0,1]$ bagi setiap $p \in P$. Sebuah item *terdeteksi* bila $c_p \geq \tau$. Himpunan item yang hilang adalah

$$M = \{p \in R \mid c_p < \tau\}. \quad (1)$$

Sebuah scan *patuh* jika dan hanya jika $M = \emptyset$, dengan skor

$$s = (|R| - |M|) / |R|. \quad (2)$$

Untuk pekerja w dengan himpunan scan S_w , tingkat kepatuhan adalah

$$C_w = |\{x \in S_w \mid M_x = \emptyset\}| / |S_w|, \quad (3)$$

dan pekerja diurutkan menurut C_w menurun; bila sama, dipakai rata-rata skor. Tingkat kepatuhan harian lokasi adalah rasio yang sama atas seluruh scan pada hari itu. R dan τ disimpan bersama setiap scan sehingga hasil lama tetap valid ketika aturan lokasi berubah.

IV. DESAIN SISTEM

A. Arsitektur berlapis

GearGuard terdiri atas empat lapisan: *domain* (tipe nilai Dart murni), *data* (detektor, penyimpanan, pembuat PDF), *state* (cubit flutter_bloc dengan state imutabel), dan *presentasi* (modul fitur). Besaran turunan— C_w , peringkat, perlengkapan yang paling sering terlewat—dihitung di lapisan state, bukan di widget, sehingga mudah diuji unit.

```
presentasi  dashboard | scanner | attendance
            team | archive | settings
state       SafetyCubit | SettingsCubit
data        PpeDetector | SafetyRepository |
report_pdf
domain      PpeItem | Employee | ScanRecord
```

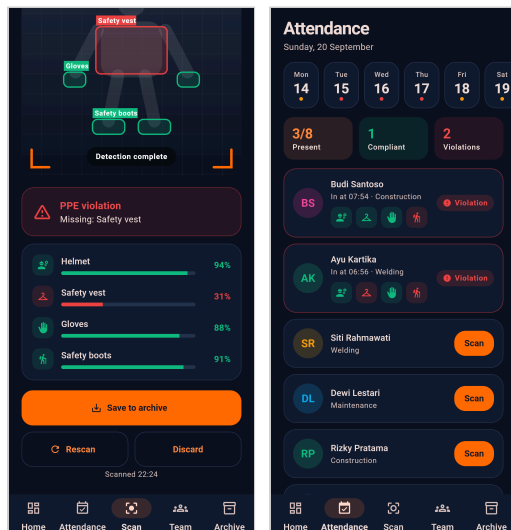
B. Antarmuka detektor

```
abstract interface class PpeDetector {
  Future<Map<PpeItem, double>> detect();
}
```

Ini satu-satunya kontrak antara model dan bagian aplikasi lainnya. Antarmuka disuntikkan di akar aplikasi, sehingga mengganti detektor simulasi dengan model pada perangkat (mis. jaringan YOLO lewat platform channel) tidak memerlukan perubahan pada layar, cubit, ataupun laporan mana pun.

C. Desain interaksi

Navigasi bawah menampilkan lima tugas harian (Home, Attendance, Scan, Team, Archive). Pelanggaran diurutkan paling atas pada daftar absensi; pekerja yang belum check-in memiliki aksi “Scan” satu ketukan; hasil scan menampilkan daftar periksa keyakinan per item dengan overlay kotak deteksi (Gbr. 1); status tidak pernah disampaikan hanya lewat warna. Tersedia tema terang dan gelap.



Gbr. 1. Hasil scan (kiri) dan tampilan absensi (kanan).

D. Penyimpanan dan pelaporan

Data karyawan dan scan disimpan sebagai JSON berversi pada penyimpanan kunci-nilai platform (seluler, desktop, dan web). Pembuat laporan menghasilkan PDF A4 bermerek berisi indikator ringkasan dan tabel per-scan untuk satu scan, satu pekerja, atau filter arsip saat ini.

V. IMPLEMENTASI DAN VERIFIKASI

Implementasi menargetkan Flutter 3.41 / Dart 3.11 dengan 32 berkas sumber ($\pm 4,9$ ribu baris) serta runner Android, iOS, web, Windows, dan macOS. Kode lolos flutter analyze tanpa error atau warning dengan aturan very_good_analysis (tersisa dua lint informasional) dan dicakup 38 tes otomatis (Bagian VI-F).

Versi saat ini memakai detektor simulasi yang mengacak keyakinan item setelah jeda singkat. Detektor ini ada untuk menjalankan seluruh alur tanpa kamera atau model terlatih; karena itu *makalah ini tidak mengklaim akurasi deteksi apa pun*. Prototipe terdahulu penulis (ppe_apk_prototype) telah menyertakan model YOLO; menghubungkannya di balik PpeDetector adalah langkah lanjutan terdekat.

E. Prototipe terdahulu penulis (fakta terukur)

Prototipe Android terdahulu penulis (ppe_apk_prototype) memakai detektor berformat YOLOv8 yang diekspor ke TFLite/ONNX (berkas TFLite 6,2 MB, didistribusikan lewat GitHub Releases) dengan enam kelas: Gloves, Vest, goggles, helmet, mask, dan safety_shoe. Inferensi berjalan native (Kotlin, pustaka Ultralytics YOLO) dengan ukuran masukan 640×640 , ambang keyakinan 0,25, dan ambang IoU NMS 0,45; data disimpan di SQLite dan laporan diekspor sebagai PDF. Repositori menyertakan 11 foto validasi (helm, rompi, sepatu keselamatan). GearGuard memetakan *safety_shoe* ke item *boots*; lima kelas lain berpadanan satu-satu.

VI. EVALUASI

A. Uji awal detektor (terukur)

Kami menjalankan model TFLite penulis (6,2 MB; masukan $1 \times 640 \times 640 \times 3$, keluaran $1 \times 10 \times 8400$) pada 11 foto validasi di repositori prototipe: lima foto helm, tiga rompi, dan tiga sepatu keselamatan, satu objek per foto. Label bersifat *lemah*: kelas yang diharapkan diambil dari nama berkas, dan tidak ada kotak kebenaran, sehingga mAP tidak dapat dihitung. Dengan pengaturan aplikasi (keyakinan 0,25; IoU NMS per-kelas 0,45), kelas yang diharapkan terdeteksi pada **11/11** foto, baik dengan resize langsung 640×640 seperti di aplikasi maupun dengan letterbox standar (Tabel I). Keyakinan untuk sepatu jauh lebih rendah pada resize langsung (0,52–0,82) dibanding letterbox (0,74–0,92), menunjukkan distorsi rasio aspek merugikan; letterbox disarankan. Latensi rata-rata 79,7 ms per frame (median 79,5; p95 82,1; 30 kali ukur setelah 3 pemanasan; LiteRT, 4 thread) pada CPU desktop AMD Ryzen (Family 25, Model 80) di Windows.

Tabel I. Deteksi kelas yang diharapkan pada 11 foto validasi.

Kelas	n	Terdeteksi	Rerata konf. (resize app)	Rerata konf. (letterbox)
helmet	5	5/5	0,89	0,89
Vest	3	3/3	0,79	0,86
safety_shoe	3	3/3	0,64	0,82
Semua	11	11/11	0,80	0,87

Keterbatasan. Sebelas foto positif saja dengan satu objek: perilaku positif-palsu tidak terukur; goggles, gloves, dan mask tidak diuji; foto berasal dari repositori prototipe sendiri dan

mungkin beririsan dengan data latih (tidak diketahui); dan latensi CPU desktop bukan latensi ponsel. Ini pemeriksaan fungsional, *bukan* estimasi akurasi. Skrip dan hasil mentah ada di repositori (tool/eval, docs/eval).

B. Sensitivitas ambang keyakinan

Tabel II menunjukkan berapa dari 11 foto yang masih menghasilkan kelas yang diharapkan ketika ambang τ dinaikkan. Letterbox mempertahankan 11/11 hingga $\tau = 0,7$ dan 10/11 pada 0,8, sedangkan resize langsung aplikasi turun ke 9/11 pada $\tau = 0,6$. Pada $\tau = 0,9$ recall runtuh (5/11 dan 3/11). Lokasi yang menginginkan aturan ketat harus menimbang perlengkapan yang terlewat terhadap alarm palsu; bawaan kami 0,6 mempertahankan 11/11 dengan letterbox.

Tabel II. Foto (dari 11) dengan kelas yang diharapkan terhadap τ .

τ	0,25	0,40	0,50	0,60	0,70	0,80	0,90
resize app	11	11	11	9	9	8	3
letterbox	11	11	11	11	11	10	5

C. Ketahanan terhadap gangguan

Setiap foto diberi gangguan (berbenih tetap; letterbox, $\tau = 0,25$) lalu dinilai ulang (Tabel III). Detektor tidak peka terhadap penggelapan, kompresi JPEG, rotasi $\pm 15^\circ$, derau sensor, dan pengecilan $4\times$ (11/11). Kelebihan cahaya kuat (10/11), blur berat (8/11 pada radius 8), dan terutama tambalan abu-abu acak yang menutup 25% citra (5/11, rerata keyakinan 0,30) menurunkan hasil. Oklusi adalah mode kegagalan lapangan yang paling relevan (tas, pekerja lain, peralatan) dan tidak dimodelkan oleh aturan satu-frame pada Bagian III.

Tabel III. Pengaruh gangguan (n = 11).

Gangguan	Terdeteksi/11	Rerata konf.	Kotak salah kelas
original	11	0,87	0
dark x0.4	11	0,87	0
bright x1.8	10	0,77	0
blur r=3	11	0,85	1
blur r=8	8	0,59	1
JPEG q=20	11	0,87	0
noise sigma=0.08	11	0,88	1
rotate +15 deg	11	0,87	0
rotate -15 deg	11	0,85	0
occlusion 25%	5	0,30	1
downscale x0.25	11	0,87	0

D. Citra negatif

Pada 15 citra non-APD (dua foto alam, enam warna polos, enam citra derau, satu gradien) model menghasilkan 2 deteksi pada $\tau = 0,25$ dan 1 pada $\tau = 0,5$ dan 0,7. Kedua positif-palsu adalah *Vest* pada bidang warna polos: oranye-merah (RGB 212, 57, 14) dengan keyakinan 0,81 dan merah muda (240, 159, 174) dengan 0,39. Foto alam tidak menghasilkan apa

pun. Ini menunjukkan bias warna terhadap rona berdaya-lihat-tinggi, sehingga ambang saja tidak dapat menghilangkan kasus oranye, dan data latih hard-negative atau augmentasi warna disarankan. Himpunan ini kecil dan sintesis; ia menunjukkan mode kegagalan, bukan laju positif-palsu.

E. Latensi

Latensi menurun seiring jumlah thread CPU (Tabel IV), dari 242,6 ms pada satu thread menjadi 70,2 ms pada delapan (50 kali ukur setelah 3 pemanasan, CPU desktop). Latensi ponsel belum diukur.

Tabel IV. Latensi inferensi terhadap jumlah thread.

Thread	Rerata (ms)	p95 (ms)
1	242,6	247,3
2	137,5	141,0
4	87,0	89,0
8	70,2	72,6

F. Verifikasi perangkat lunak

Rangkaian uji kini berisi 38 tes otomatis (unit, state, pembuatan PDF, persistensi, dan uji widget menyeluruh) dengan cakupan baris **75,1%** (1.532 dari 2.040 baris; domain 100%, data 95%, core 92%, state 80%, layar fitur 67%). Dua puluh pemeriksaan aksesibilitas (pedoman target-ketuk, target-berlabel, dan kontras teks WCAG milik Flutter pada lima layar $\times$ terang/gelap) lolos. Pemeriksaan ini awalnya *gagal* pada tiga layar tema terang, dengan kontras serendah 2,13:1: teks oranye pada permukaan oranye transparan. Perbaikannya menggelapkan primary tema terang dari #E85D00 menjadi #A83600 (6,1:1 pada permukaan) dan menurunkan warna teks status secara algoritmik, setelah itu semua pemeriksaan lolos. Build web rilis memiliki skrip utama 3,2 MB.

G. Studi kebergunaan (belum dilakukan)

Belum ada studi pengguna, sehingga tidak ada klaim kebergunaan. Protokol, kuesioner System Usability Scale dwibahasa, dan skrip penilaian tersedia di repositori (docs/paper/study); hasil sebaiknya ditafsirkan terhadap rerata acuan 68 (SD 12,5) [6]. Pemeriksaan aksesibilitas otomatis di atas bukan pengganti pengujian dengan supervisor.

VII. KETERBATASAN

Kepatuhan ditentukan per item dari satu frame; oklusi, pencahaya, dan banyak pekerja dalam satu frame belum dimodelkan. Identitas dipilih dari daftar, bukan dikenali otomatis. Penyimpanan bersifat lokal pada perangkat. Pemantauan pekerja berbasis kamera menimbulkan isu privasi dan persetujuan; penerapan harus menetapkan kebijakan retensi dan persetujuan serta sebaiknya menyimpan skor, bukan gambar. Di Indonesia, pemrosesan citra pekerja yang dapat diidentifikasi tunduk pada Undang-Undang Pelindungan Data Pribadi [7], yang mensyaratkan dasar hukum dan pembatasan tujuan.

VIII. KESIMPULAN

Kami memaparkan GearGuard, arsitektur agnostik-detektor dan implementasi sumber terbuka untuk alur kepatuhan APD. Pekerjaan selanjutnya: detektor nyata pada perangkat di balik antarmuka yang sama, studi kebergunaan lapangan, dan pelaporan tingkat departemen.

PERNYATAAN PENULIS

Karya penulis tunggal Ahmad Fauzan Prayogi (konsep, perangkat lunak, analisis, penulisan). Alat AI generatif digunakan untuk membantu penyusunan perangkat lunak dan naskah; penulis telah meninjau seluruh isi dan bertanggung jawab atasnya.

DAFTAR PUSTAKA

1. Z. Wang, Y. Wu, L. Yang, A. Thirunavukarasu, C. Evison, Y. Zhao, "Fast Personal Protective Equipment Detection for Real Construction Sites Using Deep Learning Approaches," *Sensors*, vol. 21, no. 10, Art. 3478, 2021, doi: 10.3390/s21103478.
2. J. Karlsson, F. Strand, J. Bigun, F. Alonso-Fernandez, K. Hernandez-Diaz, F. Nilsson, "Visual Detection of Personal Protective Equipment and Safety Gear on Industry Workers," arXiv:2212.04794, 2022.
3. C.-Y. Wang, A. Bochkovskiy, H.-Y. M. Liao, "YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 7464–7475.
4. G. Jocher, A. Chaurasia, J. Qiu, "Ultralytics YOLOv8," software, v8.0.0, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
5. J. Ordick, G. H. Wibowo, A. Fahmi, I. Kurniawan, E. S. Haq, "YOLOv11-Based Automated PPE Detection System for Workplace Safety Monitoring in Electric Power Distribution Operations," *Journal of Information Systems and Informatics*, vol. 7, no. 4, pp. 4294–4329, 2025, doi: 10.63158/journalisi.v7i4.1379.
6. M. Hyzy, R. Bond, M. Mulvenna, L. Bai, A. Dix, S. Leigh, S. Hunt, "System Usability Scale Benchmarking for Digital Health Apps: Meta-analysis," *JMIR mHealth and uHealth*, vol. 10, no. 8, e37290, 2022, doi: 10.2196/37290.
7. Republic of Indonesia, "Law No. 27 of 2022 on Personal Data Protection (Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi)," 2022.

Verifikasi setiap referensi pada halaman penerbit sebelum pengiriman.